Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Брандыс В.Е.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 31.10.26

Москва, 2026

Постановка задачи

Вариант 4:

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска программы. Отсортировать массив целых чисел при помощи TimSort.

Общий метод и алгоритм решения

Использованные функции:

- int strcmp(const char *str1, const char *str2); — сравнивает строки для определения названия переданного аргумента.
- int sscanf(const char *str, const char *format, ...); — преобразует строковые аргументы командной строки в числовые значения.
- void *malloc(size_t size); — выделяет динамическую память для массивов, стеков runs и аргументов потоков.
- void free(void *ptr); — освобождает ранее выделенную динамическую память.
- int pthread_create(...); — создаёт новый поток выполнения, которому передаётся функция work.
- int pthread_join(pthread_t thread, void **retval); — ожидает завершения указанного потока.
- void *work(void *ptr); — функция потока, выполняющая TimSort над своим участком массива.
- int clock_gettime(clockid_t clock_id, struct timespec *tp); — получает время для измерения длительности последовательной и параллельной сортировки, а также этапа слияния.

Описание работы программы

Программа сортирует массив целых чисел двумя способами: последовательно в одном потоке и параллельно с использованием заданного количества потоков. В основе лежит алгоритм TimSort - гибридный алгоритм сортировки, сочетающий сортировку вставками и сортировку слиянием. Параллельная версия разбивает массив на блоки, каждый поток сортирует свой блок алгоритмом TimSort, после чего выполняется финальное слияние отсортированных блоков.

1) Инициализация программы и обработка аргументов командной строки

- Инициализируются переменные size, n_threads и arr_marker, отвечающие за размер массива, количество потоков и флаг вывода массива.
- Проверяется корректность количества переданных аргументов командной строки (ожидается ровно 6 аргументов после имени программы (argc = 7).
- С помощью strcmp() определяется тип каждого аргумента: -s, -t или -a.

- С помощью sscanf() строковые значения преобразуются в числовые параметры.
- Выполняется проверка корректности параметров: size > 0, n_threads > 0, arr_marker = {0, 1}.
- При обнаружении некорректных аргументов программа выводит сообщение об ошибке и завершает работу.

2) Последовательная сортировка TimSort

- Создаётся структура ThreadArgs, содержащая указатель на массив, начальный и конечный индексы обрабатываемого участка.
- С помощью malloc() выделяется копия исходного массива (test_arr) для корректного сравнения результатов.
- С помощью clock_gettime() фиксируется время начала вычислений.
- Вызывается функция work(&serial), которая выполняет сортировку последовательно в основном потоке.
- Внутри timSort():
    - вычисляется minrun функцией getMinrun() – минимальная длина run, при которой выгодно использовать сортировку вставками;
    - в цикле вызывается createRun(), которая находит возрастающий или убывающий run (убывающий разворачивается через reverseRun()), при необходимости расширяет его до minrun сортировкой вставками (insertionSort());
    - найденный run помещается в стек stack, после чего вызывается checkAndMerge() для проверки инвариантов TimSort и при необходимости слияния (mergeRuns());
    - по завершении цикла оставшиеся runs сливаются попарно до полной сортировки массива.
- После обработки всех элементов фиксируется время окончания вычислений и рассчитывается продолжительность последовательной работы.

3) Параллельная сортировка

- С помощью malloc() выделяется память для массива идентификаторов потоков threads и массива структур с параметрами thread_args.
- Проверяется успешность выделения памяти. При ошибке программа освобождает выделенные ресурсы и завершает работу.
- Размер массива делится между потоками примерно поровну:

  bucket = size / n_threads;

  left_elements = size % n_threads;

  Оставшиеся элементы распределяются по одному между первыми потоками.

- Дополнительно проверяется, что размер блока не меньше minrun: если bucket < minrun, количество потоков уменьшается до size / minrun, чтобы каждый поток обрабатывал осмысленный по размеру участок.
- Для каждого потока задаются указатель на общий массив, начальный и конечный индексы его блока.
- С помощью clock_gettime() фиксируется время начала параллельных вычислений.
- В цикле вызывается pthread_create(), создающий отдельные потоки для выполнения функции work. Каждый поток независимо сортирует свой блок алгоритмом TimSort.
- Последний блок обрабатывается непосредственно основным потоком посредством вызова work(&thread_args[n_threads - 1]). Таким образом, основной поток также участвует в вычислениях.
- После завершения вычислений основного потока вызывается pthread_join() для каждого созданного потока, чтобы дождаться завершения их работы.
- Фиксируется время окончания параллельных вычислений и рассчитывается продолжительность параллельной работы.

Если создать один из потоков не удаётся, программа выводит сообщение об ошибке, дожидается завершения уже созданных потоков, освобождает выделенную память и завершает работу.

4) Финальное слияние блоков

- После завершения работы всех потоков формируется глобальный стек global_stack из n_threads ранов – по одному на каждый отсортированный блок.
- С помощью clock_gettime() отдельно фиксируется время этапа слияния.
- В цикле вызывается mergeRuns() для попарного слияния соседних ранов до тех пор, пока в стеке не останется один run, содержащий весь отсортированный массив.
- Этап слияния выполняется последовательно в основном потоке.
- Фиксируется время окончания слияния и вычисляется merge_time.

5) Оценка эффективности параллельных вычислений

- Вычисляется ускорение параллельного алгоритма относительно последовательного:

$$S = \frac{T_s}{T_p}$$

где T_s – время последовательного выполнения, T_p – время параллельного выполнения.

- Вычисляется эффективность использования потоков:

$$X = \frac{S}{p}$$

где p – количество ядер.

- С помощью printf() выводятся исходный и отсортированный массивы (при arr_marker = 1), время последовательной сортировки, время параллельной сортировки, время слияния, общее параллельное время, ускорение и эффективность параллельной обработки.

Код программы

```
 #include <unistd.h>

#include <pthread.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <stdint.h>

#include <stdbool.h>

#include <fcntl.h>

#include <time.h>


#define MAX_STACK_SIZE 1000


typedef struct {

    int start;

    int len;

} run;


typedef struct {

    int *arr;

    int start_idx;

    int end_idx;

} ThreadArgs;


int getMinrun(int n) {

    int r = 0;

    while (n >= 64) {

        r |= (n & 1);

        n >>= 1;
```

```
    }
    return n + r;
}

void reverseRun(int *arr, int start, int end) {
    while (start < end) {
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++;
        end--;
    }
}

void insertionSort(int* arr, int left, int right, int start) {
    for (int i = start; i <= right; ++i) {
        int temp = arr[i];
        int j = i - 1;
        while (j >= left && arr[j] > temp) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
    }
}

int createRun(int *arr, int size, int cur, int minrun) {

    if (cur >= size - 1) return size;

    int run_start = cur;
    int run_end = cur + 1;
```

```
    if (arr[run_start] <= arr[run_end]) {
        while (run_end + 1 < size && arr[run_end] <= arr[run_end + 1]) {
            run_end++;
        }
    }
    else {
        while (run_end + 1 < size && arr[run_end] > arr[run_end + 1]) {
            run_end++;
        }
        reverseRun(arr, run_start, run_end);
    }

    int run_len = run_end - run_start + 1;

    if (run_len < minrun) {
        int target_end = run_start + minrun - 1;
        if (target_end >= size) {
            target_end = size - 1;
        }
        insertionSort(arr, run_start, target_end, run_end + 1);
        run_end = target_end;
    }

    return run_end + 1;
}

void mergeRuns(int* arr, run* stack, int* stack_size, int idx) {
    run run1 = stack[idx];
    run run2 = stack[idx + 1];

    int* temp = (int*)malloc(run1.len * sizeof(int));
    if (temp == NULL) {
        printf("Memory error!\n");
```

```
        return;
    }

    for (int i = 0; i < run1.len; ++i) {
        temp[i] = arr[run1.start + i];
    }

    int cur_temp = 0;
    int cur_run2 = run2.start;
    int end_temp = run1.len;
    int end_run2 = run2.start + run2.len;
    int cur_final = run1.start;

    while (cur_temp < end_temp && cur_run2 < end_run2) {
        if (temp[cur_temp] <= arr[cur_run2]) {
            arr[cur_final++] = temp[cur_temp++];
        }
        else {
            arr[cur_final++] = arr[cur_run2++];
        }
    }

    while (cur_temp < end_temp) {
        arr[cur_final++] = temp[cur_temp++];
    }

    free(temp);
    stack[idx].len = run1.len + run2.len;
    for (int j = idx + 1; j < (*stack_size) - 1; ++j) {
        stack[j] = stack[j + 1];
    }
    (*stack_size)--;
}
```

```
void checkAndMerge(int* arr, run *stack, int* stack_size) {
    while (*stack_size > 1) {
        int n = *stack_size;

        if (n == 2) {
            int X = stack[1].len;
            int Y = stack[0].len;

            if (Y <= X) {
                mergeRuns(arr, stack, stack_size, 0);
            }
            else {
                break;
            }
        }
        else {
            int X = stack[n - 1].len;
            int Y = stack[n - 2].len;
            int Z = stack[n - 3].len;

            if (Z <= Y + X || Y <= X) {
                if (Z < X) {
                    mergeRuns(arr, stack, stack_size, n - 3);
                }
                else {
                    mergeRuns(arr, stack, stack_size, n - 2);
                }
            }
            else {
                break;
            }
        }
```

```
    }

}

void timSort(int* arr, int size) {
    if (size <= 1) return;

    run stack[MAX_STACK_SIZE];
    int stack_size = 0;

    int minrun = getMinrun(size);
    int current = 0;

    while (current < size) {
        int run_start = current;
        int next_current = createRun(arr, size, current, minrun);
        int run_len = next_current - run_start;

        stack[stack_size].start = run_start;
        stack[stack_size].len = run_len;
        stack_size++;

        checkAndMerge(arr, stack, &stack_size);

        current = next_current;
    }

    while (stack_size > 1) {
        mergeRuns(arr, stack, &stack_size, stack_size - 2);
    }
}

void *work(void *_args)
```

```
{
    ThreadArgs *args = (ThreadArgs *)_args;

    int *arr = args->arr;
    int start_idx = args->start_idx;
    int end_idx = args->end_idx;

    int local_size = end_idx - start_idx + 1;
    if (local_size <= 0) return NULL;

    timSort(&(arr[start_idx]), local_size);

    return NULL;
}

int main(int argc, char **argv) {
    // ./threads_lab -s <arrSize> -t <n_threads> -a <1/0>

    int size = 0;
    int n_threads = 0;
    int arr_marker = 0;

    if (argc != 7) {
        printf("Your input must look like:\n ./threads_lab -s <arrSize> -t <n_threads> -a <1/0>\n");
        return 1;
    }

    for (int i = 1; i < argc; i += 2) {
        if (strcmp(argv[i], "-s") == 0) {
            if (sscanf(argv[i + 1], "%d", &size) != 1 || size <= 0) {
                printf("Wrong array size parameter: %s.\n", argv[i + 1]);
                return 1;
            }
```

```
    } else if (strcmp(argv[i], "-t") == 0) {
        if (sscanf(argv[i + 1], "%d", &n_threads) != 1 || n_threads <= 0) {
            printf("Wrong threads number parameter: %s.\n", argv[i + 1]);
            return 1;
        }
    } else if (strcmp(argv[i], "-a") == 0) {
        if (sscanf(argv[i + 1], "%d", &arr_marker) != 1 || (arr_marker != 1 && arr_marker != 0)) {
            printf("Wrong array output marker parameter: %s.\n", argv[i + 1]);
            return 1;
        }
    }

    else {
        printf("Unknown key: %s\n", argv[i]);
        return 1;
    }
}

int *arr = (int*)malloc(size * sizeof(int));
if (arr == NULL) {
    printf("Memory error\n");
    free(arr);
    return 1;
}

srand((unsigned int)time(NULL));
for (int i = 0; i < size; ++i) {
    arr[i] = (rand() % 65536) - 32768;
}

if (arr_marker == 1) {
    printf("Original array:\n");
    for (int i = 0; i < size; ++i) {
```

```
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int start_idx = 0;
int *test_arr = (int*)malloc(size * sizeof(int));
for (int i = 0; i < size; ++i) {
    test_arr[i] = arr[i];
}
if (test_arr == NULL) {
    printf("Memory error\n");
    free(test_arr);
    return 1;
}

struct timespec start, end;

ThreadArgs serial = (ThreadArgs){
    .arr = test_arr,
    .start_idx = start_idx,
    .end_idx = size - 1,
};

clock_gettime(CLOCK_MONOTONIC, &start);
work(&serial);
clock_gettime(CLOCK_MONOTONIC, &end);
double serial_time = (end.tv_sec - start.tv_sec) * 1e3 + (end.tv_nsec - start.tv_nsec) / 1e6;

printf("Serial sort:\n");
if (arr_marker == 1) {
    for (int i = 0; i < size; ++i) {
        printf("%d ", test_arr[i]);
```

```
    }
    printf("\n");
}
printf("time: %f ms\n", serial_time);

int bucket = size / n_threads;
int left_elements = size % n_threads;
int minrun = getMinrun(size);

if (bucket < minrun) {
    n_threads = size / minrun;
    bucket = size / n_threads;
    left_elements = size % n_threads;
}

pthread_t *threads = (pthread_t*)malloc(n_threads * sizeof(pthread_t));
ThreadArgs *thread_args = (ThreadArgs*)malloc(n_threads * sizeof(ThreadArgs));
if (threads == NULL || thread_args == NULL) {
    printf("Memory error\n");
    free(threads);
    free(thread_args);
    return 1;
}

for (int i = 0; i < size; ++i) {
    test_arr[i] = arr[i];
}

for (int i = 0; i < n_threads; ++i) {
    int bucket_size = bucket + (left_elements > 0 ? 1 : 0);
    left_elements--;

    thread_args[i] = (ThreadArgs){
```

```
        .arr = test_arr,
        .start_idx = start_idx,
        .end_idx = start_idx + bucket_size - 1,
    };

    start_idx += bucket_size;
}

printf("Parallel sort:\n");
clock_gettime(CLOCK_MONOTONIC, &start);

int threads_created = 0;
for (int i = 0; i < n_threads - 1; ++i) {
    if (pthread_create(&threads[i], NULL, work, &thread_args[i]) != 0) {
        printf("Error while creating thread.\n");
        for (int j = 0; j < threads_created; ++j) {
            pthread_join(threads[j], NULL);
        }
        free(threads);
        free(thread_args);
        return 1;
    }
    ++threads_created;
}

work(&(thread_args[n_threads - 1]));

run *global_stack = (run*)malloc(n_threads * sizeof(run));
int global_stack_size = 0;

for (int i = 0; i < threads_created; ++i) {
    pthread_join(threads[i], NULL);
}
```

```
for (int i = 0; i < n_threads; ++i) {
    run adding_run;
    adding_run.start = thread_args[i].start_idx;
    adding_run.len = thread_args[i].end_idx - thread_args[i].start_idx + 1;
    global_stack[global_stack_size++] = adding_run;
}

free(threads);
free(thread_args);

clock_gettime(CLOCK_MONOTONIC, &end);
double parallel_time = (end.tv_sec - start.tv_sec) * 1e3 + (end.tv_nsec - start.tv_nsec) / 1e6;

clock_gettime(CLOCK_MONOTONIC, &start);
while (global_stack_size > 1) {
    mergeRuns(test_arr, global_stack, &global_stack_size, global_stack_size - 2);
}
clock_gettime(CLOCK_MONOTONIC, &end);

double merge_time = (end.tv_sec - start.tv_sec) * 1e3 + (end.tv_nsec - start.tv_nsec) / 1e6;
double total_parallel_time = parallel_time + merge_time;

if (arr_marker == 1) {
    for (int i = 0; i < size; ++i) {
        printf("%d ", test_arr[i]);
    }
    printf("\n");
}

double speedup = serial_time / total_parallel_time;
double efficiency = speedup / n_threads;
```

```
    printf("work time: %f ms; merge time: %f ms; time: %f ms\n", parallel_time, merge_time, total_parallel_time);
    printf("speedup: %f; efficiency: %f\n", speedup, efficiency * 100.0);
    free(global_stack);
    free(arr);
    free(test_arr);

    return 0;
}
```

Протокол работы программы

Тестирование:

Проверка, что сортировка рабочая:

awareness@mine:~/threads$ ./threads_lab -t 1 -s 10 -a 1

Original array:

21 32143 4950 20885 31090 -26977 18647 12430 -19136 -379

Serial sort:

-26977 -19136 -379 21 4950 12430 18647 20885 31090 32143

Parallel sort:

-26977 -19136 -379 21 4950 12430 18647 20885 31090 32143

Тестирование потоков:

awareness@mine:~/threads$ ./threads_lab -t 2 -s 10000000 -a 0

Serial sort:

time: 1442.720715 ms

Parallel sort:

work time: 767.874380 ms; merge time: 38.281082 ms; time: 806.155462 ms

speedup: 1.789631; efficiency: 89.481544

awareness@mine:~/threads$ ./threads_lab -t 4 -s 10000000 -a 0

Serial sort:

time: 1410.896809 ms

Parallel sort:

work time: 370.924146 ms; merge time: 66.775559 ms; time: 437.699705 ms

speedup: 3.223436; efficiency: 80.585890

awareness@mine:~/threads$ ./threads_lab -t 8 -s 10000000 -a 0

Serial sort:

time: 1426.894464 ms

Parallel sort:

work time: 213.593101 ms; merge time: 109.214435 ms; time: 322.807536 ms

speedup: 4.420264; efficiency: 55.253297

awareness@mine:~/threads$ ./threads_lab -t 12 -s 10000000 -a 0

Serial sort:

time: 1447.710179 ms

Parallel sort:

work time: 148.303816 ms; merge time: 167.777890 ms; time: 316.081706 ms

speedup: 4.580177; efficiency: 38.168142

awareness@mine:~/threads$ ./threads_lab -t 16 -s 10000000 -a 0

Serial sort:

time: 1432.106084 ms

Parallel sort:

work time: 119.862929 ms; merge time: 210.725442 ms; time: 330.588371 ms

speedup: 4.331992; efficiency: 27.074948

awareness@mine:~/threads$ ./threads_lab -t 128 -s 10000000 -a 0

Serial sort:

time: 1414.446666 ms

Parallel sort:

work time: 113.773122 ms; merge time: 1491.461608 ms; time: 1605.234730 ms

speedup: 0.881146; efficiency: 0.688396

awareness@mine:~/threads$ ./threads_lab -t 256 -s 10000000 -a 0

Serial sort:

time: 1420.274533 ms

Parallel sort:

work time: 104.151178 ms; merge time: 2916.877242 ms; time: 3021.028420 ms

speedup: 0.470129; efficiency: 0.183644

awareness@mine:~/threads$ ./threads_lab -t 512 -s 10000000 -a 0

Serial sort:

time: 1433.260015 ms

Parallel sort:

work time: 117.758835 ms; merge time: 5610.995450 ms; time: 5728.754285 ms

speedup: 0.250187; efficiency: 0.048865

awareness@mine:~/threads$ ./threads_lab -t 1024 -s 10000000 -a 0

Serial sort:

time: 1405.653242 ms

Parallel sort:

work time: 159.416512 ms; merge time: 11222.232482 ms; time: 11381.648994 ms

speedup: 0.123502; efficiency: 0.012061

Таблица тестирования:

Количество потоков	Время параллельной сортировки	Общее время параллельного выполнения	Ускорение	Эффективность (%)
2	767.874380	806.155462	1.789631	89.481544
4	370.924146	437.699705	3.223436	80.585890
8	213.593101	322.807536	4.420264	55.253297
12	148.303816	316.081706	4.580177	38.168142
16	119.862929	330.588371	4.331992	27.074948
128	113.773122	1605.234730	0.881146	0.688396
256	104.151178	3021.028420	0.470129	0.183644
512	117.758835	5728.754285	0.250187	0.048865
1024	159.416512	11381.648994	0.123502	0.012061

График зависимости времени выполнения от количества потоков:

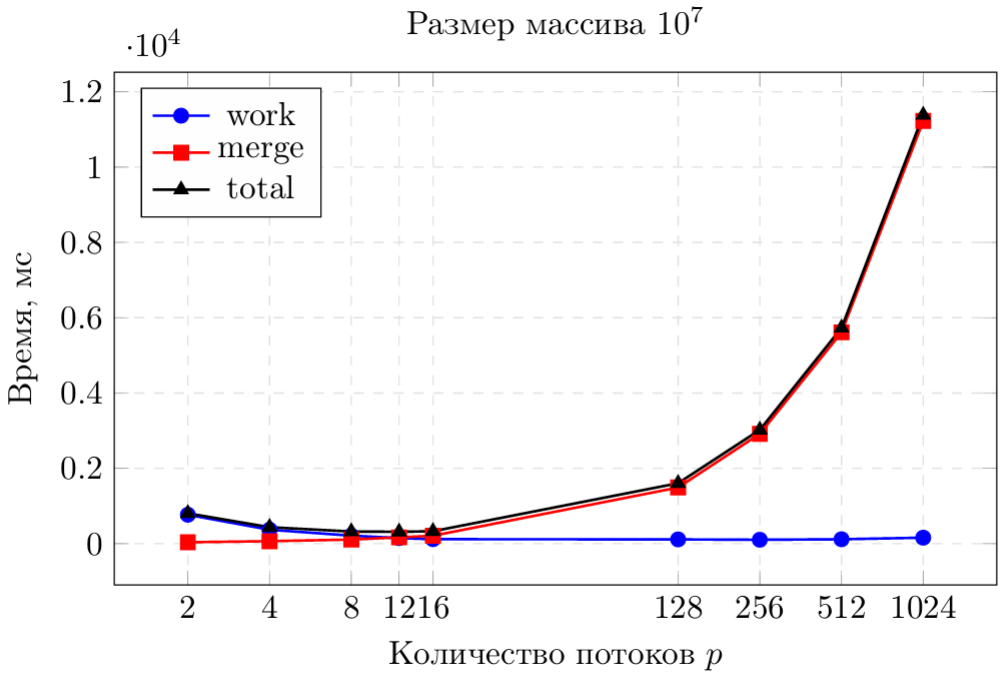
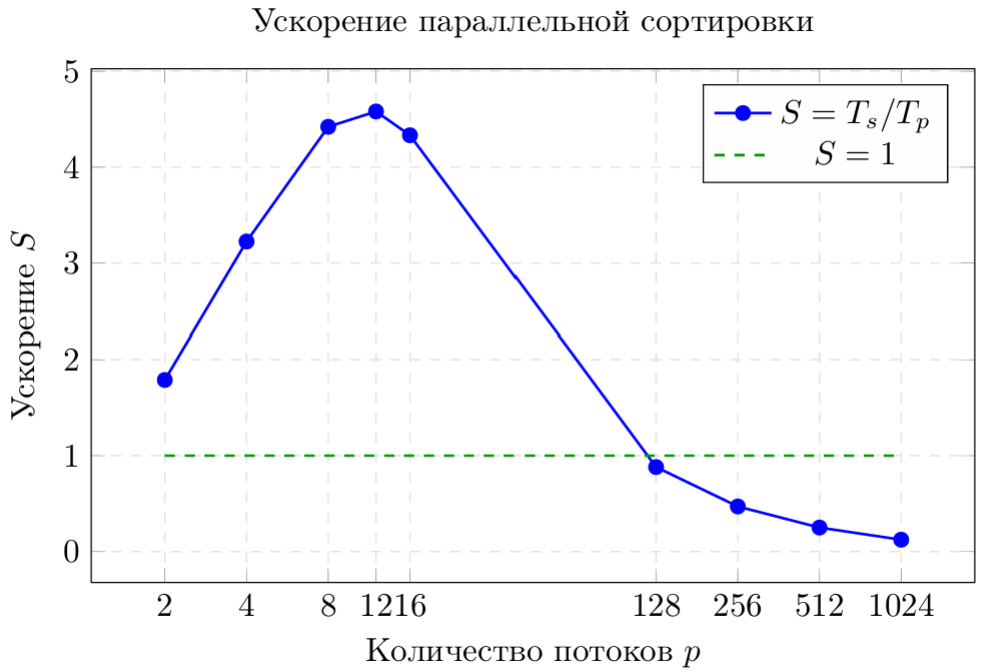


График зависимости ускорения от количества потоков:



Вывод

В ходе лабораторной работы была успешно реализована программа на языке C для параллельной сортировки данных алгоритмом TimSort с использованием библиотеки pthread. Программа сортирует массив целых чисел как последовательно, так и с использованием нескольких потоков. Были реализованы распределение массива между потоками, измерение времени выполнения отдельных этапов (параллельная сортировка блоков и финальное слияние), а также расчёт ускорения и эффективности параллельных вычислений.

В результате экспериментов установлено, что увеличение количества потоков позволяет сократить время работы на этапе сортировки блоков, однако этап финального слияния выполняется последовательно, и его доля в общем времени растёт с увеличением числа потоков (так как увеличивается количество блоков для слияния). Поэтому максимальное ускорение ограничено: при большом числе потоков время слияния начинает доминировать, и дальнейшее увеличение их числа не приводит к ускорению, а снижает производительность. В результате было установлено, что оптимальное количество потоков зависит от особенностей алгоритма и его реализации.